

FACULTY OF ENGINEERING
Alexandria University

Simple Communication System

Project Report

Simulation in MATLAB

Course	Signals and Systems
Department	Electronics and Communication Engineering
Academic Year	Spring 2026

Submitted by:

Student Name	ID Number
Amr Mahmoud Moussa	23010637
Yasen Mohamed Ahmed Salhin	23010962

1. Introduction

This project implements a simple communication system using MATLAB. The main idea is to transmit an audio signal through a communication channel, add noise to simulate real-world transmission conditions, and then recover the signal using a receiver filter.

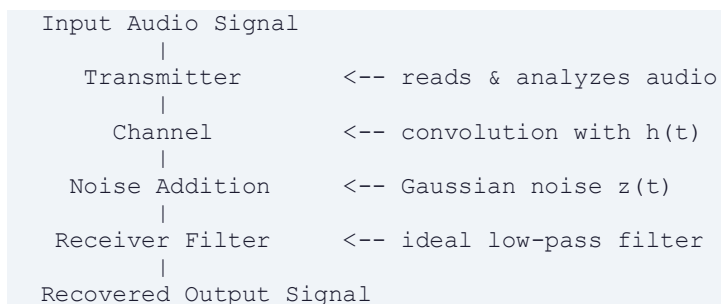
The system consists of four main stages:

1. Transmitter
2. Channel
3. Noise Addition
4. Receiver (Ideal Low-Pass Filter)

The project requires playing the sound file, plotting it in both time and frequency domains, passing it through different channel impulse responses, adding Gaussian noise, filtering the noisy signal, and comparing the results for speech and music signals.

2. System Block Diagram

The communication system can be represented as the following pipeline:



The transmitter prepares and analyzes the input audio signal. The channel modifies the signal according to its impulse response. Noise is then added to simulate real channel interference. Finally, the receiver uses an ideal low-pass filter to reduce the noise effect.

3. Transmitter Stage

In the transmitter stage, the input audio file is read using MATLAB:

```
[x, Fs] = audioread('speech.wav');
```

If the audio file is stereo, it is converted to mono:

```
if size(x,2) > 1
    x = mean(x, 2);
end
```

The signal is then forced to be a column vector for consistent processing:

```
x = x(:);
```

The number of samples and the time vector are computed as:

```
N = length(x);  
t = (0:N-1)/Fs;
```

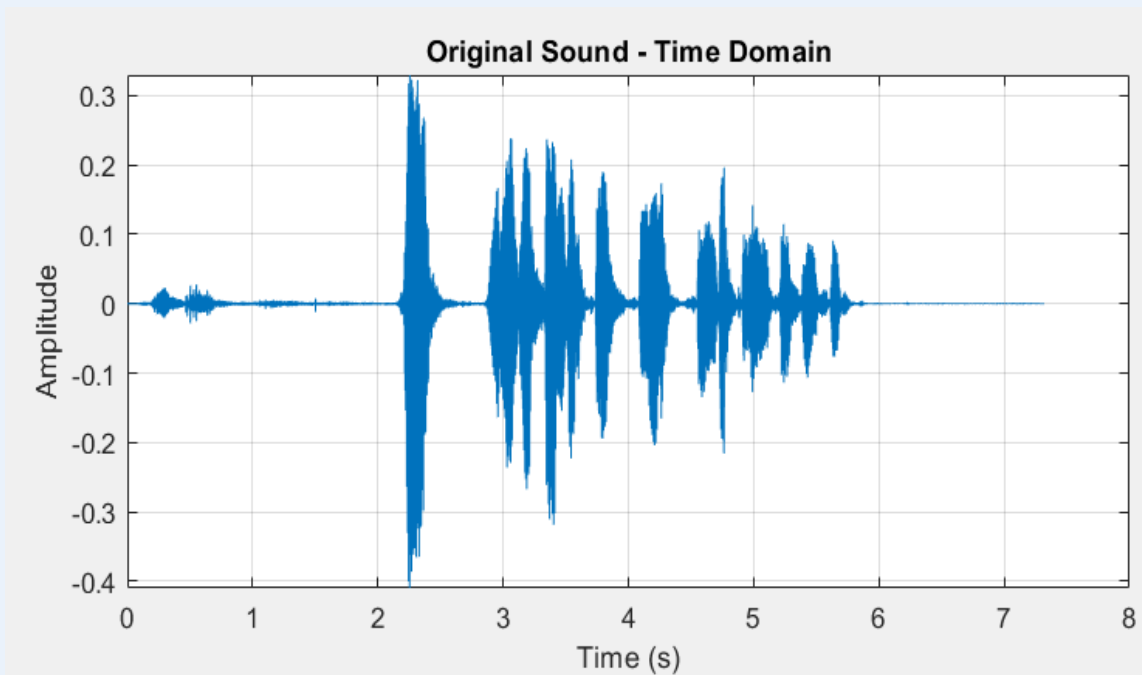
Since the sampling frequency F_s represents the number of samples per second, each sample is separated by the sampling period $T_s = 1/F_s$. The time vector converts sample indices into real time values in seconds.

4. Time-Domain Representation

The time-domain plot shows how the amplitude of the sound signal changes with time:

```
plot(t, x)  
title('Original Sound - Time Domain')  
xlabel('Time (s)')  
ylabel('Amplitude')
```

The horizontal axis represents time in seconds, and the vertical axis represents the signal amplitude. The speech signal appears as a varying waveform. High-amplitude regions correspond to louder parts of the audio, while low-amplitude regions represent silence or softer speech.



5. Frequency-Domain Representation

The signal is converted to the frequency domain using FFT:

```
X = fftshift(fft(x));  
f = (-N/2:N/2-1)*(Fs/N);
```

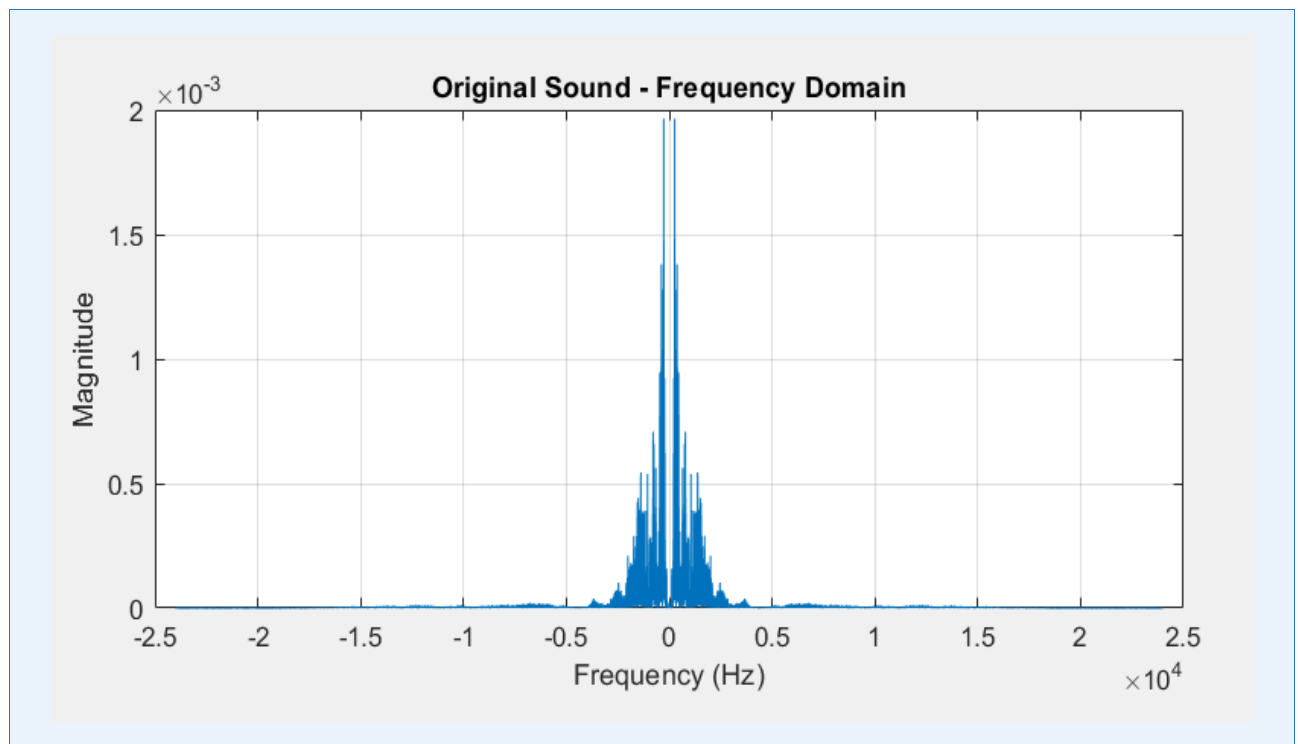
The FFT shows the frequency components that form the signal. `fftshift` rearranges the spectrum so that zero frequency is centered, with negative frequencies on the left and positive frequencies on the right.

$$\Delta f = F_s / N \quad (\text{frequency resolution})$$

The magnitude spectrum is plotted and normalized by dividing by N so the spectrum does not depend on the number of samples:

```
plot(f, abs(X)/N)
```

The speech signal contains most of its useful frequency components at relatively low frequencies. This is why a low-pass filter with cutoff frequency around 3400 Hz can still preserve speech intelligibility.



6. Channel Stage

The channel modifies the transmitted signal according to its impulse response. In communication systems, the output of a channel is obtained by convolution:

$$y(t) = x(t) * h(t)$$

where $x(t)$ is the input signal, $h(t)$ is the channel impulse response, and $y(t)$ is the channel output. The project uses four different channel impulse responses. The channel time vector is defined as:

```
t_h = 0:1/Fs:1;
```

6.1 Channel 1: Delta Function

```
h1 = zeros(size(t_h));  
h1(1) = 1;
```

This represents $h_1(t) = \delta(t)$. The delta channel is ideal — no memory, no delay, no distortion. Using the convolution property: $x(t) * \delta(t) = x(t)$, so the output is essentially identical to the input.

Expected result: The output y_1 looks and sounds almost the same as the original audio.

6.2 Channel 2: Fast Exponential Decay

```
h2 = exp(-2*pi*5000*t_h);
```

This represents $h_2(t) = e^{(-2\pi*5000*t)}$. The channel responds strongly at first and then decays very quickly, causing only mild smoothing. Because the decay is fast, distortion is small.

Expected result: y_2 becomes slightly smoother with mild attenuation of high-frequency details.

6.3 Channel 3: Slow Exponential Decay

```
h3 = exp(-2*pi*1000*t_h);
```

This represents $h_3(t) = e^{(-2\pi*1000*t)}$. This exponential decays more slowly — the channel has longer memory. Previous samples affect the output for a longer time, causing stronger smoothing and more distortion than Channel 2.

Expected result: y_3 appears more blurred with larger amplitude due to accumulation from more previous samples.

6.4 Channel 4: Echo Channel

```
h4 = zeros(size(t_h));  
h4(1) = 2;  
h4(end) = 0.5;
```

This represents $h_4(t) = 2\delta(t) + 0.5\delta(t-1)$. The signal arrives immediately with gain 2, and another weaker copy arrives after 1 second, giving output: $y_4(t) = 2x(t) + 0.5x(t-1)$.

Expected result: The output contains the original signal plus a delayed weaker version, producing an echo effect similar to sound reflections in a room or wireless multipath propagation.

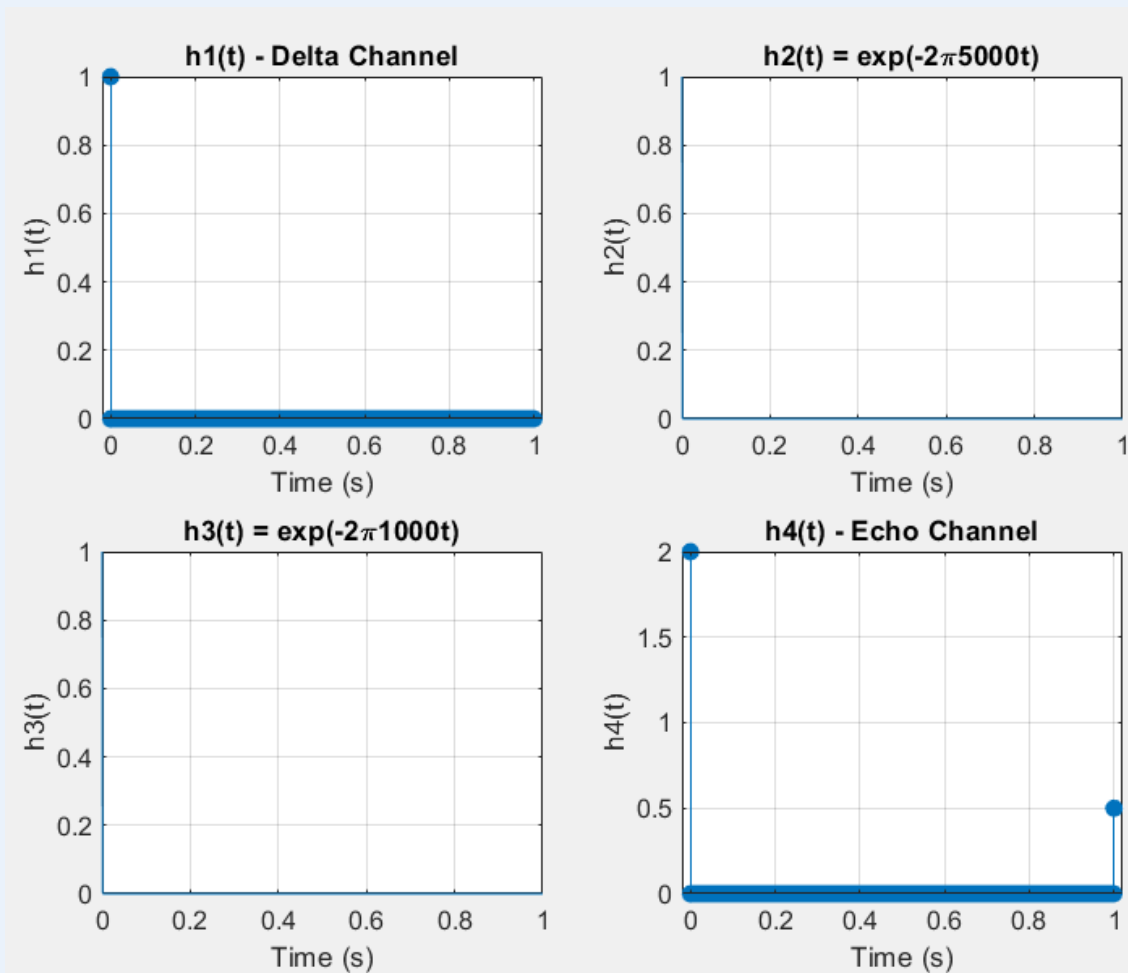
7. Passing the Signal Through the Channels

The signal is passed through each channel using convolution with the 'same' option to keep output length equal to input length:

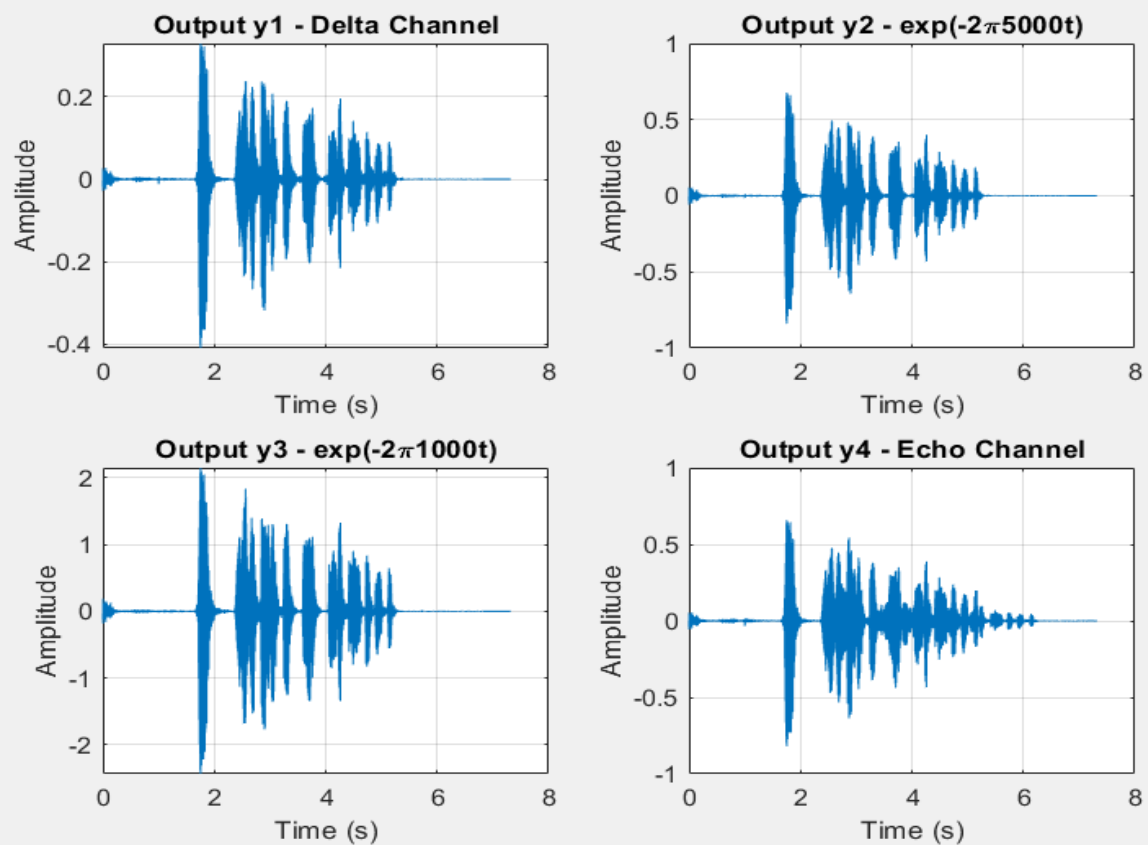
```
y1 = conv(x, h1, 'same');
y2 = conv(x, h2, 'same');
y3 = conv(x, h3, 'same');
y4 = conv(x, h4, 'same');
```

For playback, the signals are normalized to the range $[-1, +1]$ to prevent clipping distortion:

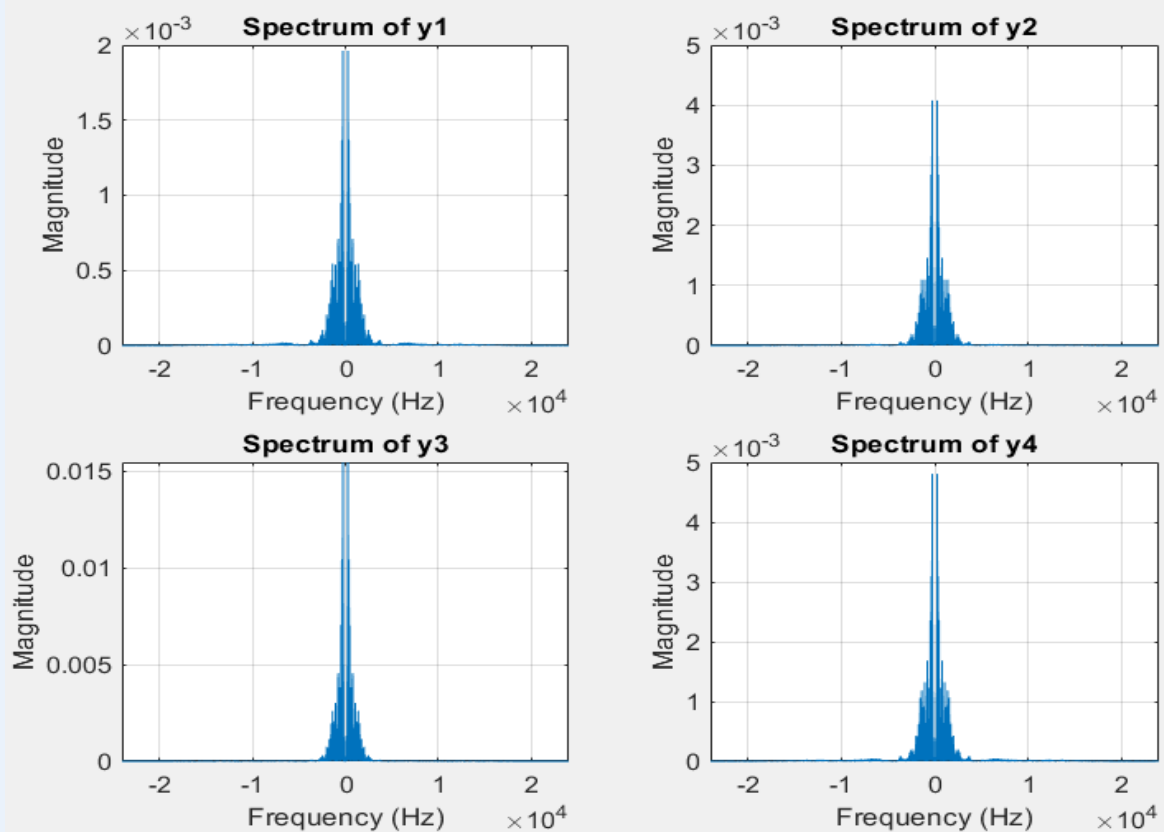
```
y1_play = y1 / max(abs(y1));
y2_play = y2 / max(abs(y2));
y3_play = y3 / max(abs(y3));
y4_play = y4 / max(abs(y4));
```



Four channels impulse responses



Four channels outputs — Time Domain



Four channels outputs — Frequency Domain

8. Channel Comparison

Channel	Main Effect	Practical Meaning
Delta Function	No change	Ideal perfect channel
$e^{(-2\pi*5000*t)}$	Mild smoothing	Short-memory channel
$e^{(-2\pi*1000*t)}$	Stronger smoothing	Long-memory channel
Echo Channel	Delayed copy added	Reflection / multipath channel

The exponential channels behave like low-pass systems because they smooth fast changes in the signal. Channel 3 causes more distortion than Channel 2 because its impulse response decays more slowly, giving it longer memory.

9. Noise Addition

After selecting one channel output, Gaussian noise is added:

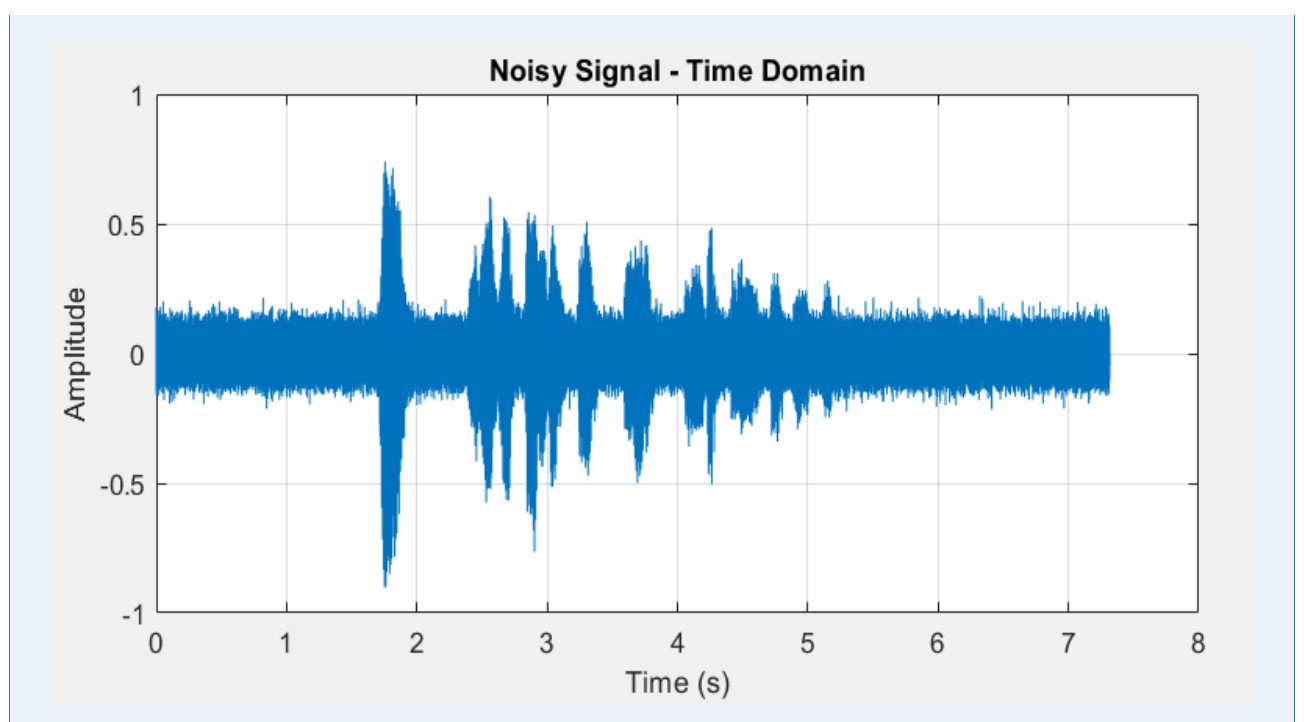
```
sigma = 0.05;

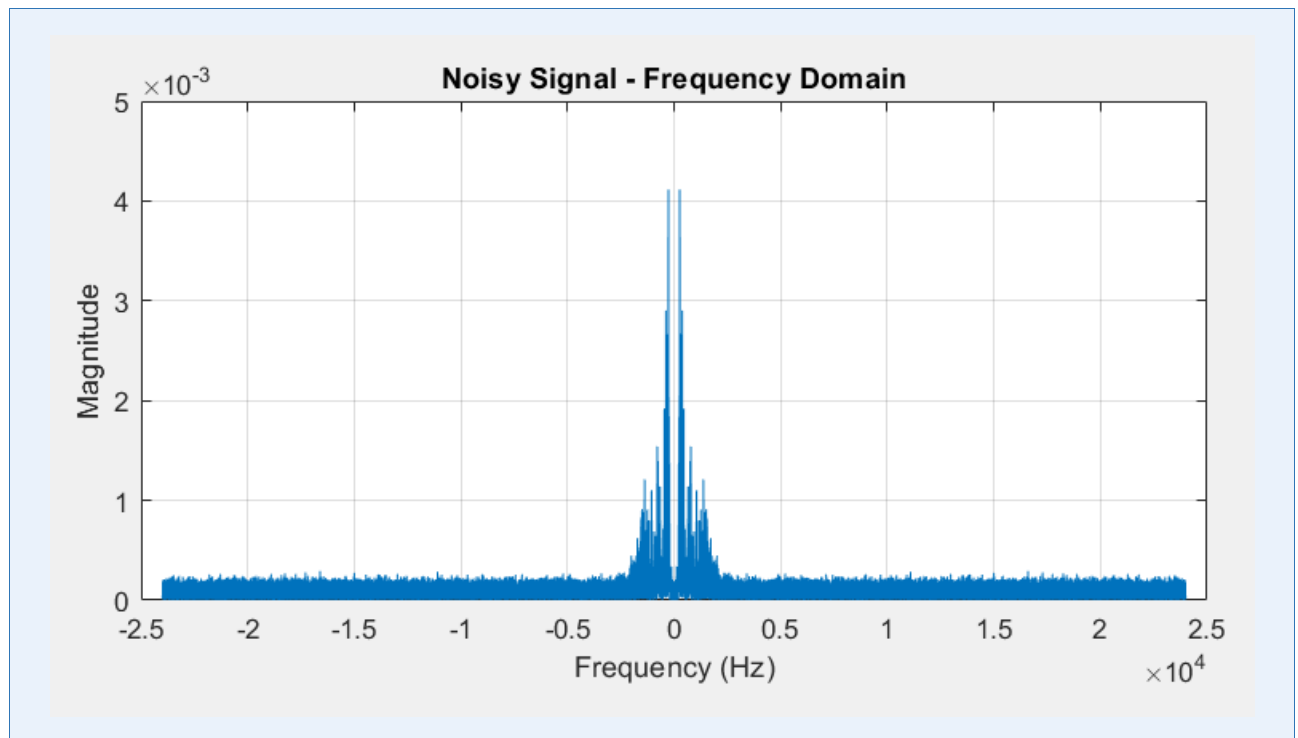
z = sigma * randn(size(selected_channel));
noisy_signal = selected_channel + z;
```

The noise equation is $z(t) = \text{sigma} * \text{randn}$, where sigma controls the noise strength. Small values produce weak noise; large values produce strong noise. The noisy signal is:

$$y_{\text{noisy}}(t) = y(t) + z(t)$$

Since noise varies rapidly from sample to sample, it introduces many frequency components, especially high-frequency ones. This is why the noisy signal spectrum becomes wider and more random than the clean signal spectrum.





10. Receiver Stage: Ideal Low-Pass Filter

The receiver uses an ideal low-pass filter with cutoff frequency 3400 Hz:

```
cutoff = 3400;  
H_lpf = abs(f) <= cutoff;
```

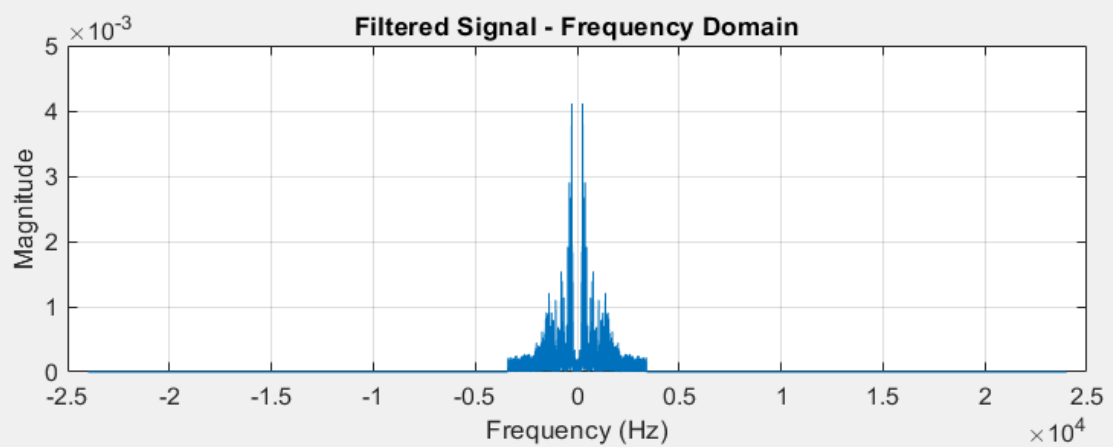
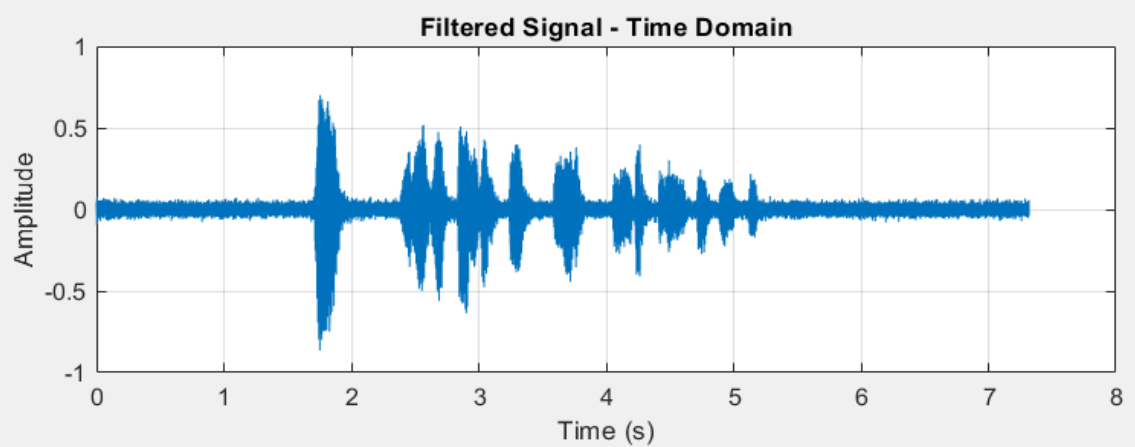
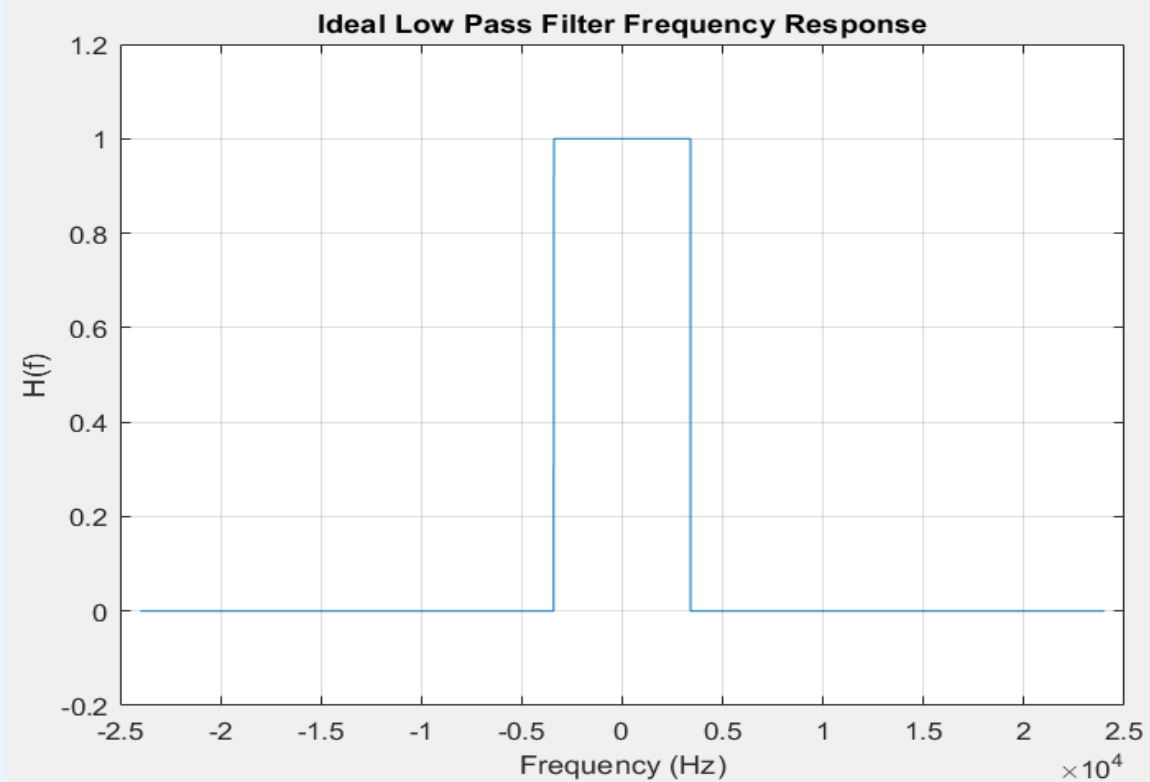
This creates a filter that keeps frequencies between -3400 Hz and +3400 Hz and removes everything outside this range:

$$H(f) = 1 \quad \text{if } |f| \leq 3400$$
$$H(f) = 0 \quad \text{if } |f| > 3400$$

The filter is applied in the frequency domain and the signal is then converted back to the time domain:

```
Filtered_F = Noisy_F .* H_lpf. '  
filtered_signal = real(ifft(ifftshift(Filtered_F)));
```

`ifftshift` reverses the earlier `fftshift`, `ifft` converts back to time domain, and `real()` removes tiny imaginary numerical errors.



Filtered Signal

11. Effect of the Receiver Filter

After filtering, the signal becomes smoother and the high-frequency noise is reduced. However, the filter cannot remove noise that lies inside the speech frequency band. Therefore, the filtered signal is improved but not perfectly identical to the original signal.

The filtered sound is clearer than the noisy sound, but it may sound slightly muffled because some high-frequency components are also removed.

12. Speech vs. Music Comparison

12.1 Speech Signal

Speech is mainly concentrated in low and mid frequencies. Therefore, using a 3400 Hz low-pass filter preserves most of the important speech information. Speech remains understandable after filtering, although it may sound slightly less sharp.

12.2 Music Signal

Music contains a wider range of frequencies than speech, including vocals, instruments, harmonics, and high-frequency details. When the same 3400 Hz low-pass filter is applied, many high-frequency components are removed. The music quality decreases more noticeably than speech and may sound muffled or less rich.

13. Saved Output Files

The MATLAB code saves three output audio files:

```
audiowrite('channel_output.wav', selected_channel / max(abs(selected_channel)),  
Fs);  
audiowrite('noisy_output.wav', noisy_signal / max(abs(noisy_signal)),  
Fs);  
audiowrite('filtered_output.wav', filtered_signal / max(abs(filtered_signal)),  
Fs);
```

14. Conclusion

In this project, a simple communication system was implemented in MATLAB. The audio signal was first analyzed in both time and frequency domains, then passed through four different channel impulse responses.

The delta channel gave almost ideal transmission with no major change. The exponential channels caused smoothing because of channel memory, with $(e^{-2\pi 1000t})$ producing stronger distortion than $(e^{-2\pi 5000t})$. The fourth channel produced an echo effect due to the delayed impulse component.

Gaussian noise was added to simulate real channel noise, which made the signal more distorted and spread extra components across the frequency spectrum. At the receiver, an ideal low-pass filter with a cutoff frequency of 3400 Hz reduced high-frequency noise and improved the sound quality.

The low-pass filter worked better for speech than music because speech mainly depends on lower frequencies, while music contains more high-frequency details.

Overall, the project demonstrates the main concepts of communication systems: signal transmission, channel distortion, noise addition, frequency-domain filtering, and signal recovery.